GC - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4410 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 4393 values"
## [1] "17 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	548	35	132	NA	NA	715
2	66	NA	434	100	110	710
3	316	NA	171	208	16	711
4	304	NA	244	68	37	653
5	237	NA	214	73	39	563
6	209	1	139	73	23	445
7	179	NA	105	29	18	331
8	93	NA	56	26	4	179
9	77	NA	2	NA	7	86

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7664336	0.0489510	0.1846154	NA	NA	715
2	0.0929577	NA	0.6112676	0.1408451	0.1549296	710
3	0.4444444	NA	0.2405063	0.2925457	0.0225035	711
4	0.4655436	NA	0.3736600	0.1041348	0.0566616	653
5	0.4209591	NA	0.3801066	0.1296625	0.0692718	563
6	0.4696629	0.0022472	0.3123596	0.1640449	0.0516854	445
7	0.5407855	NA	0.3172205	0.0876133	0.0543807	331
8	0.5195531	NA	0.3128492	0.1452514	0.0223464	179
9	0.8953488	NA	0.0232558	NA	0.0813953	86

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

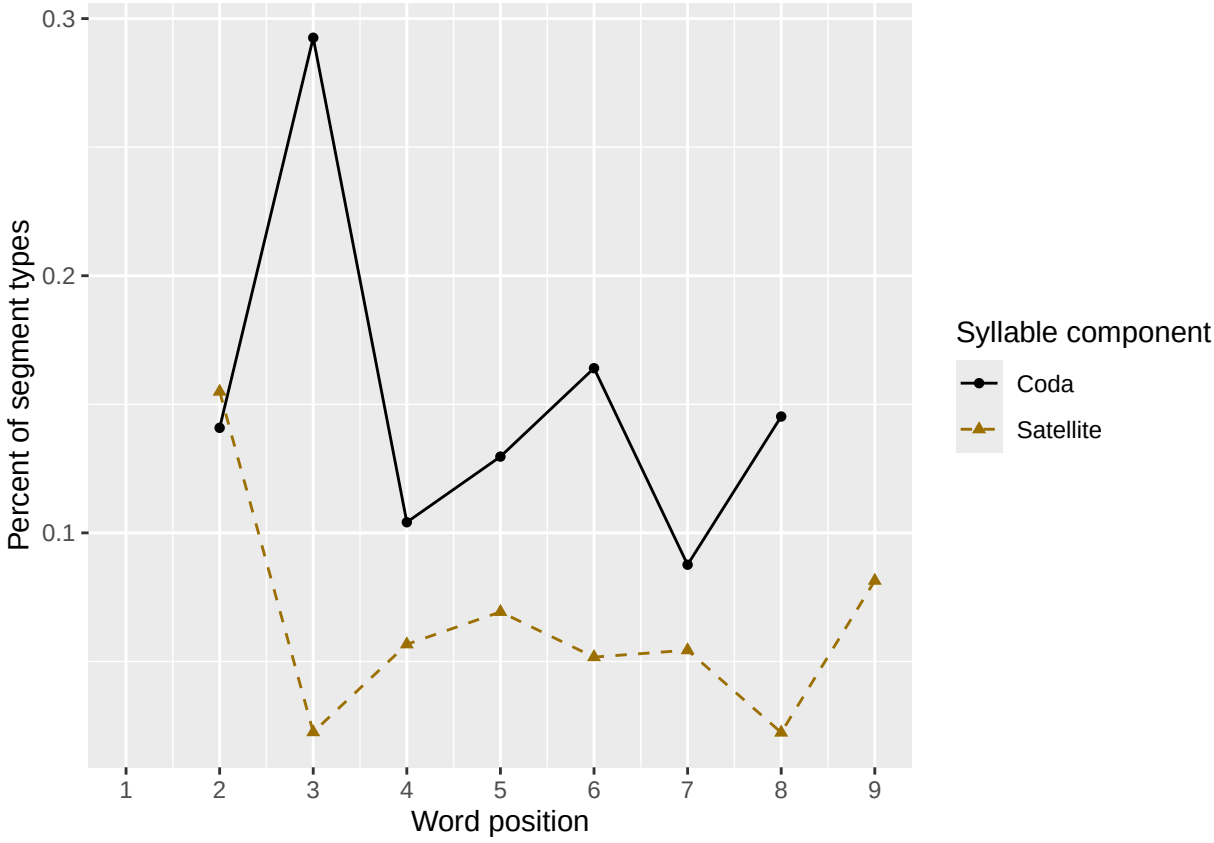
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.966 1     0.948 NA     NA     NA     NA     NA     NA
## 2       5 0.826 0.967 0.880  0.978 NA     NA     NA     NA     NA
## 3       6 0.849 0.924 0.950  0.932  0.958 NA     NA     NA     NA
## 4       7 0.833 0.938 0.902  0.921  0.947  0.965 NA     NA     NA
## 5       8 0.856 0.947 0.907  0.887  0.941  0.915  0.961 NA     NA
## 6       9 0.892 0.968 0.925  0.935  0.903  0.946  0.946  0.968 NA
## 7      10 0.895 1     0.884  0.849  0.942  0.919  0.953  0.930  0.988
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

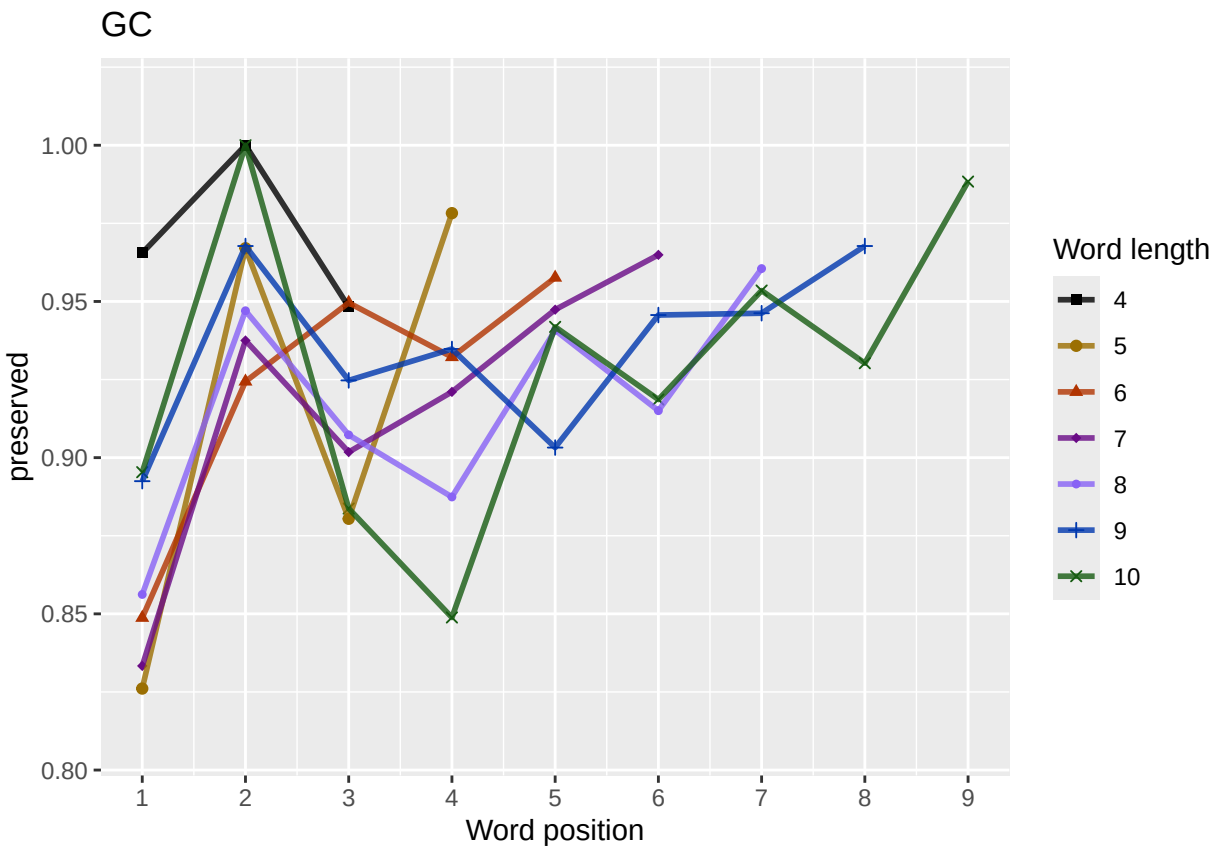
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    58    58    58    NA    NA    NA    NA    NA    NA
## 2       5    92    91    92    92    NA    NA    NA    NA    NA
## 3       6   119   119   119   118   118    NA    NA    NA    NA
## 4       7   114   112   112   114   114   114    NA    NA    NA
## 5       8   153   151   151   151   152   153   152    NA    NA
## 6       9    93    93    93    92    93    92    93    93    NA
## 7      10    86    86    86    86    86    86    86    86    86
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.17841           0.22763          -0.16453           1.59608           0.02022          -0
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2313  AIC: 2325
## log likelihood:  -1156.381
## Nagelkerke R2:  0.01935483
## % pres/err predicted correctly:  -606.2266
## % of predictable range [ (model-null)/(1-null) ]:  0.007994504
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     1.72884      0.02338      0.41827     -0.03120
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2318  AIC: 2326
## log likelihood:  -1159.069
## Nagelkerke R2:  0.01642171
## % pres/err predicted correctly:  -606.9226
## % of predictable range [ (model-null)/(1-null) ]:  0.006857509
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     2.44246     -0.06354      0.15403
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2321  AIC: 2327
## log likelihood:  -1160.393
## Nagelkerke R2:  0.01497516
## % pres/err predicted correctly:  -607.405
## % of predictable range [ (model-null)/(1-null) ]:  0.00606931
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
```

```
##       2.016        0.137
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2328
## log likelihood:  -1161.997
## Nagelkerke R2:  0.01322138
## % pres/err predicted correctly:  -607.796
## % of predictable range [ (model-null)/(1-null) ]:  0.005430539
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.41782     -0.06285     -0.00170      0.16758
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2321  AIC: 2329
## log likelihood:  -1160.384
## Nagelkerke R2:  0.01498444
## % pres/err predicted correctly:  -607.3804
## % of predictable range [ (model-null)/(1-null) ]:  0.006109651
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    1.955624    -0.005146     0.178486
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2330
## log likelihood:  -1161.918
## Nagelkerke R2:  0.01330815
## % pres/err predicted correctly:  -607.7029
## % of predictable range [ (model-null)/(1-null) ]:  0.005582794
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.507
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4392 Residual
```

```
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2350
## log likelihood:  -1174.054
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -611.1202
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    2.484930     0.002913
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2352
## log likelihood:  -1174.051
## Nagelkerke R2:  4.082105e-06
## % pres/err predicted correctly:  -611.1192
## % of predictable range [ (model-null)/(1-null) ]:  1.604814e-06
## ***************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2324.763	0.000000	1.000000	0.4387381	0.0193548	1.178408	2.227628	5.5960766	- 0.1808820	- 2.1645324	0.0202218
preserved ~ stimlen * pos	2326.137	1.374858	0.502868	0.2206276	0.0164217	1.728841	0.023382	8.4182665	- 0.0311973	NA	NA
preserved ~ stimlen + pos	2326.782	2.023208	0.363635	0.1595406	0.0149752	2.424455	- 0.0635398	0.1540309	NA	NA	NA
preserved ~ pos	2327.994	4.231868	0.198705	0.0871794	0.0132212	4.016021	NA	0.1369989	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2328.769	0.006214	0.1349155	0.5059192	0.6014984	0.44178181	-0.0628532	0.1675758	NA	-0.0016998	NA
preserved ~ I(pos^2) + pos	2329.835	0.073189	0.0791357	0.2967471	0.5813308	0.19556237	NA	0.1784864	NA	-0.0051456	NA
preserved ~ 1	2350.102	25.346276	0.0000003	0.0000011	0.0000000	0.25073124	NA	NA	NA	NA	NA
preserved ~ stimlen	2352.102	27.338852	0.0000001	0.0000004	0.0000000	0.14849301	-0.0029129	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.17841           0.22763          -0.16453           1.59608           0.02022          -0
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2313  AIC: 2325
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

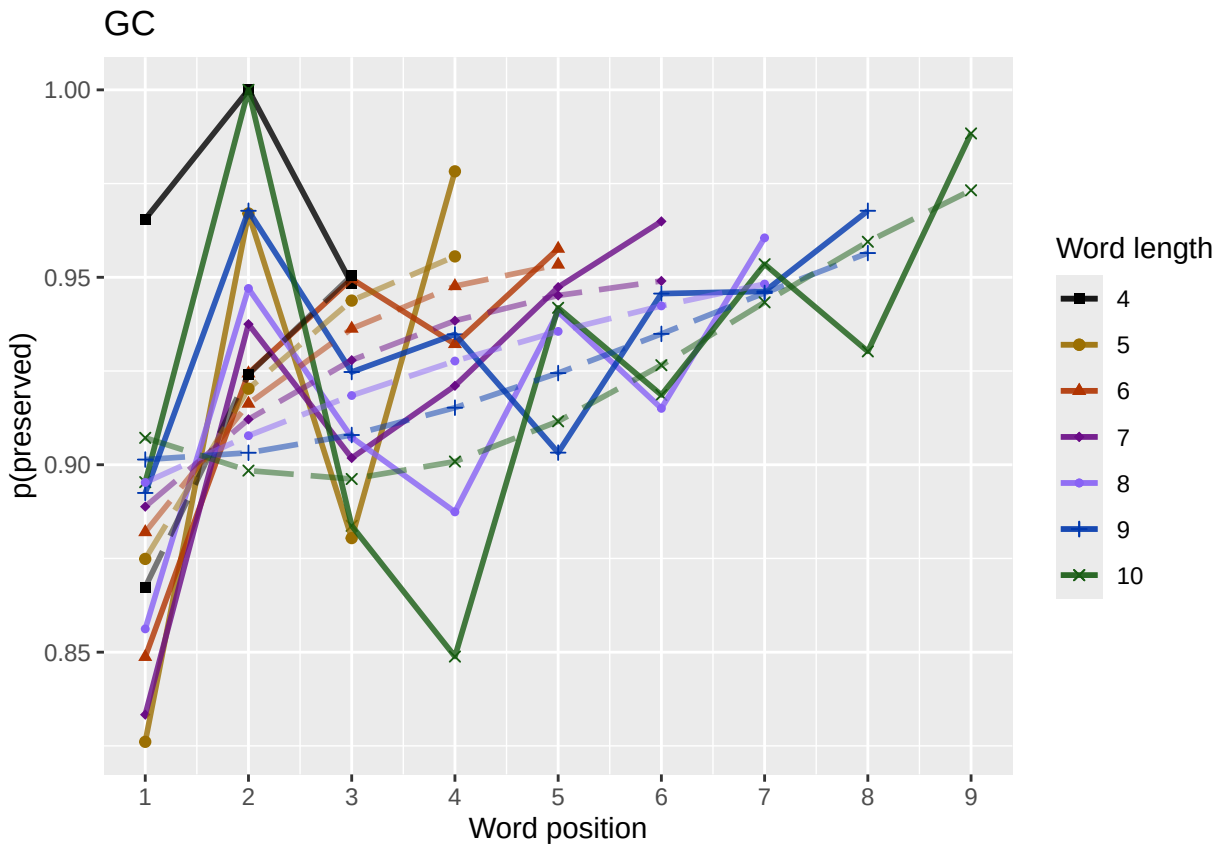
```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.867 0.924 0.950 NA     NA     NA     NA     NA     NA
## 2       5 0.875 0.920 0.944  0.956 NA     NA     NA     NA     NA
## 3       6 0.882 0.916 0.936  0.948  0.953 NA     NA     NA     NA
## 4       7 0.889 0.912 0.928  0.938  0.945  0.949 NA     NA     NA
## 5       8 0.895 0.908 0.918  0.928  0.936  0.942  0.948 NA     NA
## 6       9 0.901 0.903 0.908  0.915  0.924  0.935  0.946  0.956 NA
## 7      10 0.907 0.898 0.896  0.901  0.912  0.927  0.943  0.959  0.973
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models
```

```
# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1        7   715
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 7 / 715 = 0.98 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.40440           0.20701          -0.12125           1.36445           0.01678          -0
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4370 Residual
## Null Deviance:       2259
## Residual Deviance: 2211  AIC: 2223
## log likelihood:  -1105.323
## Nagelkerke R2:  0.02745838
## % pres/err predicted correctly:  -575.7882
## % of predictable range [ (model-null)/(1-null) ]:  0.0105494
```

```
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     2.30906     -0.05739      0.19729
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4373 Residual
## Null Deviance:       2259
## Residual Deviance: 2218  AIC: 2224
## log likelihood:  -1108.883
## Nagelkerke R2:  0.02346527
## % pres/err predicted correctly:  -576.6033
## % of predictable range [ (model-null)/(1-null) ]:  0.009151177
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     1.69142      0.01825      0.43384     -0.02812
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4372 Residual
## Null Deviance:       2259
## Residual Deviance: 2216  AIC: 2224
## log likelihood:  -1107.911
## Nagelkerke R2:  0.0245562
## % pres/err predicted correctly:  -576.2108
## % of predictable range [ (model-null)/(1-null) ]:  0.009824475
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      1.9224       0.1825
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4374 Residual
## Null Deviance:       2259
## Residual Deviance: 2220  AIC: 2224
## log likelihood:  -1110.159
## Nagelkerke R2:  0.02203289
## % pres/err predicted correctly:  -576.8941
## % of predictable range [ (model-null)/(1-null) ]:  0.008652229
## ***************************
## model index:  7
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.41789     -0.06029      0.00807      0.13488
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4372 Residual
## Null Deviance:       2259
## Residual Deviance: 2217  AIC: 2225
## log likelihood:  -1108.722
## Nagelkerke R2:  0.02364632
## % pres/err predicted correctly:  -576.7039
## % of predictable range [ (model-null)/(1-null) ]:  0.008978497
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     1.97386      0.00467      0.14603
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4373 Residual
## Null Deviance:       2259
## Residual Deviance: 2220  AIC: 2226
## log likelihood:  -1110.104
## Nagelkerke R2:  0.02209457
## % pres/err predicted correctly:  -576.9646
## % of predictable range [ (model-null)/(1-null) ]:  0.008531378
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##        2.56
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4375 Residual
## Null Deviance:       2259
## Residual Deviance: 2259  AIC: 2261
## log likelihood:  -1129.687
## Nagelkerke R2:  -5.505941e-16
## % pres/err predicted correctly:  -581.9378
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)      stimlen
##     2.38151      0.02332
##
## Degrees of Freedom: 4375 Total (i.e. Null);  4374 Residual
## Null Deviance:       2259
## Residual Deviance: 2259  AIC: 2263
## log likelihood:  -1129.459
## Nagelkerke R2:  0.0002588051
## % pres/err predicted correctly:  -581.88
## % of predictable range [ (model-null)/(1-null) ]:  9.914654e-05
## ***************************
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                        AIC=NoFrag_LPRes$AIC,
                        row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	2222.646	0.000000	1.000000	0.3360538	0.0027458	2.440440	0.2070071	0.3644464	-0.1606240	-0.1212538	0.0167777
preserved ~ stimlen + pos	2223.766	1.119990	0.571211	0.1919580	0.0023462	2.330905	0.1972920	NA	NA	NA	NA
preserved ~ stimlen * pos	2223.822	1.175945	0.555452	0.1866610	0.0024556	2.691420	0.0182493	0.4338396	-0.0281227	NA	NA
preserved ~ pos	2224.318	1.671212	0.433611	0.1457169	0.0022032	2.992240	NA	0.1825077	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	2225.442	2.797417	0.246915	0.0829770	0.0023646	2.341788	0.0602941	0.1348830	NA	0.0080699	NA
preserved ~ I(pos^2) + pos	2226.203	3.561384	0.168521	0.0566323	0.0022094	2.697386	NA	0.1460306	NA	0.0046695	NA
preserved ~ 1	2261.374	38.727781	0.000000	0.0000000	0.0000000	2.056004	NA	NA	NA	NA	NA
preserved ~ stimlen	2262.914	40.271028	0.000000	0.0000000	0.0002588	2.381514	0.0233223	NA	NA	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

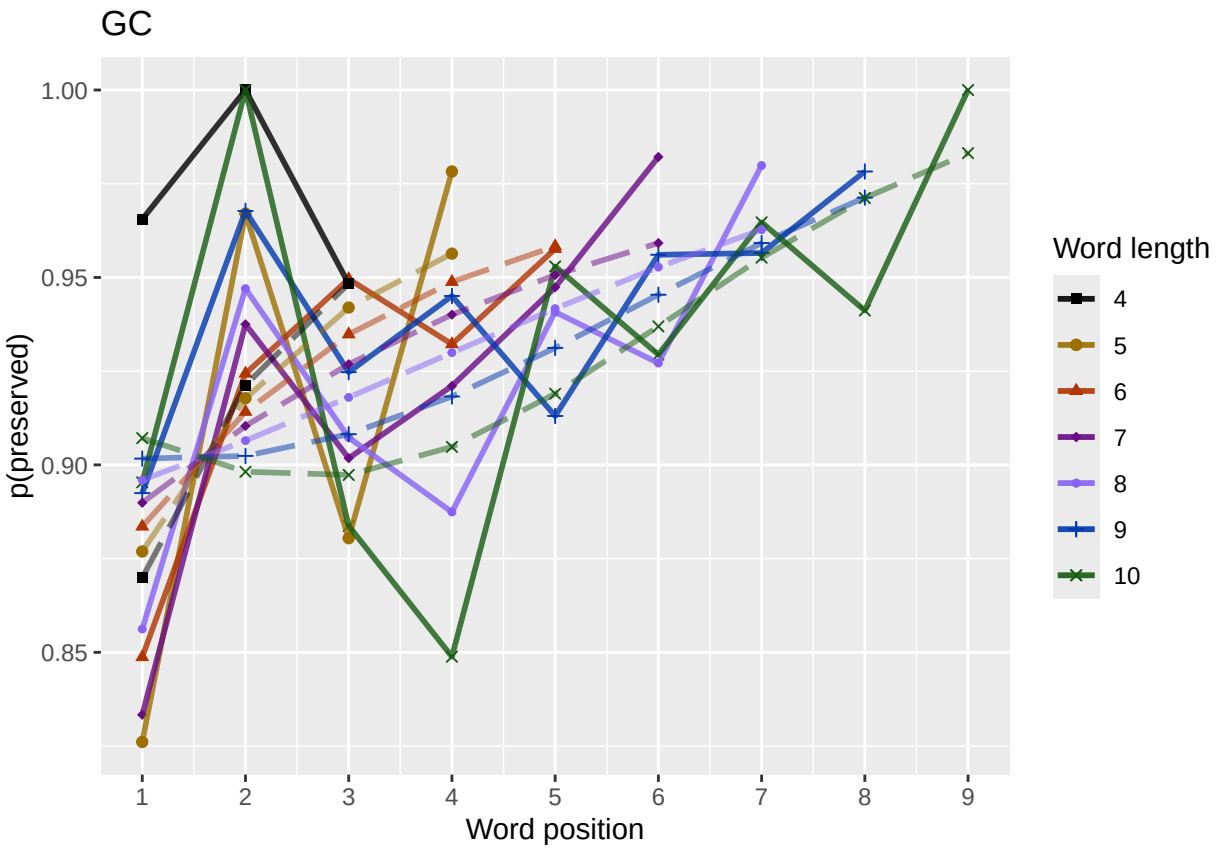
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.870 0.921 0.948 NA     NA     NA     NA     NA     NA
## 2       5 0.877 0.918 0.942  0.956 NA     NA     NA     NA     NA
## 3       6 0.884 0.914 0.935  0.949  0.958 NA     NA     NA     NA
## 4       7 0.890 0.910 0.927  0.940  0.951  0.959 NA     NA     NA
## 5       8 0.896 0.906 0.918  0.930  0.942  0.953  0.963 NA     NA
## 6       9 0.902 0.902 0.908  0.918  0.931  0.945  0.959  0.971 NA
## 7      10 0.907 0.898 0.897  0.905  0.919  0.937  0.955  0.971  0.983
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                        paste0(NoFragData$patient[1]),
                                        "LPFitted",
                                        NULL,
                                        palette_values,
                                        shape_values,
                                        obs_linetypes,
                                        pred_linetypes = c("longdash")
                                        )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.81 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] -0.004366873
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] 0.01379757
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "No U-shape in this participant"
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "no U-shape in this participant"
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
##  (Intercept)           pos      log_freq  pos:log_freq
##      2.03715       0.13749       0.23408      -0.02449
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2296  AIC: 2304
## log likelihood:  -1147.942
## Nagelkerke R2:  0.0285423
## % pres/err predicted correctly:  -602.7988
## % of predictable range [ (model-null)/(1-null) ]:  0.01359445
## ***************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq
##          1.929271          -0.009256           0.212471           0.080536          -0.013175
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2292  AIC: 2304
## log likelihood:  -1146.04
## Nagelkerke R2:  0.0306077
## % pres/err predicted correctly:  -602.5484
## % of predictable range [ (model-null)/(1-null) ]:  0.01400344
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)           pos      log_freq
##      1.9973       0.1513        0.1530
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2298  AIC: 2304
## log likelihood:  -1149.221
## Nagelkerke R2:  0.02715181
## % pres/err predicted correctly:  -603.4939
## % of predictable range [ (model-null)/(1-null) ]:  0.01245874
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##          2.11922          -0.02107           0.39993           0.15505          -0.03259
```

```
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4388 Residual
## Null Deviance:       2348
## Residual Deviance: 2295  AIC: 2305
## log likelihood:  -1147.55
## Nagelkerke R2:  0.02896774
## % pres/err predicted correctly:  -603.0538
## % of predictable range [ (model-null)/(1-null) ]:  0.01317787
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##      2.11013      -0.01090       0.14035       0.23020      -0.02408
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4388 Residual
## Null Deviance:       2348
## Residual Deviance: 2296  AIC: 2306
## log likelihood:  -1147.9
## Nagelkerke R2:  0.02858843
## % pres/err predicted correctly:  -602.823
## % of predictable range [ (model-null)/(1-null) ]:  0.01355492
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##          2.12173          -0.01697           0.41033           0.14516          -0.02630          -0
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2294  AIC: 2306
## log likelihood:  -1146.967
## Nagelkerke R2:  0.02960134
## % pres/err predicted correctly:  -602.6282
## % of predictable range [ (model-null)/(1-null) ]:  0.01387307
## ***************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           I(pos^2)               pos           log_freq  I(pos
##          1.979172          -0.006792          -0.008877           0.211210           0.078531
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4386 Residual
## Null Deviance:       2348
```

```
## Residual Deviance: 2292  AIC: 2306
## log likelihood:  -1146.024
## Nagelkerke R2:  0.03062511
## % pres/err predicted correctly:  -602.5702
## % of predictable range [ (model-null)/(1-null) ]:  0.01396786
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     2.10479     -0.01587      0.15500      0.14934
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2298  AIC: 2306
## log likelihood:  -1149.131
## Nagelkerke R2:  0.02725051
## % pres/err predicted correctly:  -603.5149
## % of predictable range [ (model-null)/(1-null) ]:  0.01242456
## ***************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)                pos   stim
##          1.998916          -0.012382           0.236010          -0.008281           0.210782
##      log_freq:pos
##          0.078981
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4385 Residual
## Null Deviance:       2348
## Residual Deviance: 2291  AIC: 2307
## log likelihood:  -1145.418
## Nagelkerke R2:  0.03128274
## % pres/err predicted correctly:  -602.4436
## % of predictable range [ (model-null)/(1-null) ]:  0.0141747
## ***************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##         2.073932         -0.019794          0.401055         -0.003117          0.179845         -0.0
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2295  AIC: 2307
```

```
## log likelihood:  -1147.522
## Nagelkerke R2:  0.02899824
## % pres/err predicted correctly:  -603.007
## % of predictable range [ (model-null)/(1-null) ]:  0.01325431
## **************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    2.072442    -0.014956    -0.002228     0.172750     0.149390
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4388 Residual
## Null Deviance:       2348
## Residual Deviance: 2298  AIC: 2308
## log likelihood:  -1149.116
## Nagelkerke R2:  0.02726623
## % pres/err predicted correctly:  -603.4853
## % of predictable range [ (model-null)/(1-null) ]:  0.01247292
## **************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.17841           0.22763          -0.16453           1.59608           0.02022          -0
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4387 Residual
## Null Deviance:       2348
## Residual Deviance: 2313  AIC: 2325
## log likelihood:  -1156.381
## Nagelkerke R2:  0.01935483
## % pres/err predicted correctly:  -606.2266
## % of predictable range [ (model-null)/(1-null) ]:  0.007994504
## **************************
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     1.72884      0.02338      0.41827     -0.03120
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2318  AIC: 2326
## log likelihood:  -1159.069
## Nagelkerke R2:  0.01642171
## % pres/err predicted correctly:  -606.9226
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.006857509
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     2.44246     -0.06354      0.15403
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2321  AIC: 2327
## log likelihood:  -1160.393
## Nagelkerke R2:  0.01497516
## % pres/err predicted correctly:  -607.405
## % of predictable range [ (model-null)/(1-null) ]:  0.00606931
## ***************************
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##       2.016        0.137
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2328
## log likelihood:  -1161.997
## Nagelkerke R2:  0.01322138
## % pres/err predicted correctly:  -607.796
## % of predictable range [ (model-null)/(1-null) ]:  0.005430539
## ***************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.41782     -0.06285     -0.00170      0.16758
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2321  AIC: 2329
## log likelihood:  -1160.384
## Nagelkerke R2:  0.01498444
## % pres/err predicted correctly:  -607.3804
## % of predictable range [ (model-null)/(1-null) ]:  0.006109651
## ***************************
## model index:  19
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    1.955624    -0.005146     0.178486
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2330
## log likelihood:  -1161.918
## Nagelkerke R2:  0.01330815
## % pres/err predicted correctly:  -607.7029
## % of predictable range [ (model-null)/(1-null) ]:  0.005582794
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
##          2.16464           0.04545           0.39810          -0.03247
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2323  AIC: 2331
## log likelihood:  -1161.299
## Nagelkerke R2:  0.01398496
## % pres/err predicted correctly:  -607.169
## % of predictable range [ (model-null)/(1-null) ]:  0.006454951
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##     2.14969      0.05069      0.14830
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2326  AIC: 2332
## log likelihood:  -1162.873
## Nagelkerke R2:  0.01226304
## % pres/err predicted correctly:  -607.7506
## % of predictable range [ (model-null)/(1-null) ]:  0.005504779
## ***************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)
##       2.507
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4392 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2350
## log likelihood:  -1174.054
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -611.1202
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    2.484930     0.002913
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2352
## log likelihood:  -1174.051
## Nagelkerke R2:  4.082105e-06
## % pres/err predicted correctly:  -611.1192
## % of predictable range [ (model-null)/(1-null) ]:  1.604814e-06
## ***************************
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                      AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	log_freq	pos	log_freq:pos	I(pos^2)	log_freq:I(pos^2)	stimlen:I(pos^2)
preserved ~ pos * log_freq	2303.884	0.000000	1.000000	0.9545128	0.2374751	0.2340771	0.1374888	NA	NA	0.0244928	NA	NA	NA

Model	AIC	Delta_AIC	AICcWt	AggR2	(Intercept)	stimlen	log_freq	pos	I(pos^2)	log_freq:pos	log_freq:I(pos^2)	pos:I(pos^2)	stimlen:log_freq	stimlen:pos	stimlen:I(pos^2)
preserved ~ (I(pos^2) + pos) * log_freq	2304.0	0.0	0.3	0.1	0.0805385	0.2124784	- 0.0092561	- 0.1131745	NA	NA	NA				
preserved ~ pos + log_freq	2304.4	0.4	0.2	0.1	0.1530142	0.1512520	NA	NA	NA	NA	NA				
preserved ~ stimlen * log_freq + pos	2305.1	1.1	0.2	0.1	0.3999254	0.1553496	NA	NA	NA	NA	NA				
preserved ~ stimlen + pos * log_freq	2305.7	1.7	0.1	0.1	0.2302003	0.1403507	NA	NA	NA	NA	NA				
preserved ~ stimlen * log_freq + pos * log_freq	2305.2	2.2	0.1	0.1	0.4103261	0.1455631	-	NA	NA	NA	NA				
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2306.2	3.2	0.1	0.1	0.0785319	0.2112087	- 0.0088771	- 0.1131383	NA	NA	NA				
preserved ~ stimlen + pos + log_freq	2306.2	3.2	0.1	0.1	0.1493147	0.1555033	NA	NA	NA	NA	NA				
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	2306.3	3.3	0.1	0.1	0.2360100	0.2107849	0.0789810	NA	- 0.0082809	NA	NA				
preserved ~ stimlen * log_freq + I(pos^2) + pos	2307.3	4.3	0.1	0.1	0.4010552	0.1798453	NA	- 0.0031166	NA	NA	NA				
preserved ~ stimlen + I(pos^2) + pos + log_freq	2308.4	5.4	0.0	0.1	0.1493900	0.1725704	NA	- 0.0022284	NA	NA	NA				
preserved ~ stimlen * (I(pos^2) + pos)	2324.7	21.7	0.0	0.1	NA	1.5961766	NA	- 0.1645324	NA	NA	- 0.1808822	0.0202218			
preserved ~ stimlen * pos	2326.2	23.2	0.0	0.1	NA	0.4182665	NA	NA	NA	NA	- 0.0311973	NA			
preserved ~ stimlen + pos	2326.7	23.7	0.0	0.1	NA	NA	0.1543409	NA	NA	NA	NA	NA			

Model	AIC DeltaAIC AICexp AICw NagR2 (Intercept) stimlen log_freq stimlen:log_freq pos log_freq:pos I(pos^2) log_freq:I(pos^2) stimlen:pos stimlen:I(pos^2) [illegible]									
preserved ~ pos	2327.[illegible]	NA	NA	0.1369[illegible]	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	2328.[illegible] -0.0628532	NA	NA	0.1675[illegible]	NA	-0.0016998	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	2329.[illegible]	NA	NA	0.1784[illegible]	NA	-0.0051456	NA	NA	NA	NA
preserved ~ stimlen * log_freq	2330.[illegible] -0.0324686	NA	NA	NA	NA	NA	NA	NA	NA	
preserved ~ stimlen + log_freq	2331.[illegible]	NA	NA	NA	NA	NA	NA	NA	NA	
preserved ~ 1	2350.[illegible]	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	2352.[illegible]	NA	NA	NA	NA	NA	NA	NA	NA	NA

```r
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos * log_freq"
```

```r
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)          pos     log_freq  pos:log_freq
##      2.03715      0.13749      0.23408      -0.02449
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2296  AIC: 2304
```

```r
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

```r
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

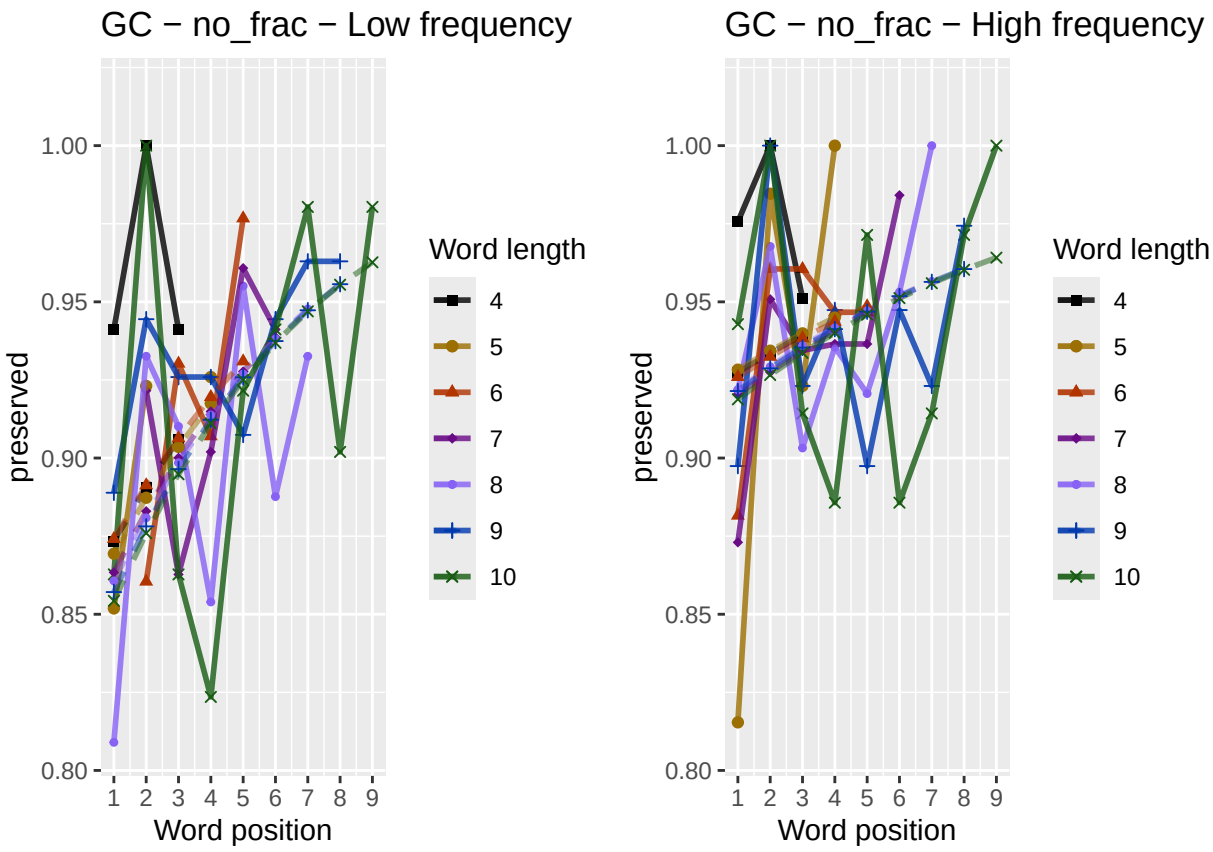
```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```




```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0394       0.2094
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2300  AIC: 2304
## log likelihood:  -1150.244
## Nagelkerke R2:  0.02603934
## % pres/err predicted correctly:  -604.7994
## % of predictable range [ (model-null)/(1-null) ]:  0.01032608
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##       2.016        0.137
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2328
## log likelihood:  -1161.997
## Nagelkerke R2:  0.01322138
## % pres/err predicted correctly:  -607.796
## % of predictable range [ (model-null)/(1-null) ]:  0.005430539
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    1.955624    -0.005146     0.178486
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2330
## log likelihood:  -1161.918
## Nagelkerke R2:  0.01330815
## % pres/err predicted correctly:  -607.7029
## % of predictable range [ (model-null)/(1-null) ]:  0.005582794
## ***************************
## model index:  2
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      2.6339      -0.4042
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2327  AIC: 2331
## log likelihood:  -1163.327
## Nagelkerke R2:  0.01176689
## % pres/err predicted correctly:  -607.1748
## % of predictable range [ (model-null)/(1-null) ]:  0.006445437
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.507
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4392 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2350
## log likelihood:  -1174.054
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -611.1202
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    2.484930     0.002913
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2352
## log likelihood:  -1174.051
## Nagelkerke R2:  4.082105e-06
## % pres/err predicted correctly:  -611.1192
## % of predictable range [ (model-null)/(1-null) ]:  1.604814e-06
## ***************************
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
```

```
                        AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                          by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	2304.489	0.00000	1.0e+00	0.9999869	0.0260393	2.039361	0.2094239	NA	NA	NA	NA
preserved ~ pos	2327.994	23.50565	7.9e-06	0.0000079	0.0132212	4.016021	NA	NA	NA	0.1369989	NA
preserved ~ (I(pos^2) + pos)	2329.836	25.34696	3.1e-06	0.0000030	0.0133081	1.955624	NA	NA	-0.0051456	0.1784864	NA
preserved ~ CumErr	2330.654	26.16499	2.1e-06	0.0000020	0.0117662	9.633851	NA	-0.4042221	NA	NA	NA
preserved ~ 1	2350.109	45.62006	0.0e+00	0.0000000	0.0000000	2.507312	NA	NA	NA	NA	NA
preserved ~ stimlen	2352.101	47.61263	0.0e+00	0.0000000	0.0000042	2.484930	NA	NA	NA	NA	0.0029129

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random average"),
```

```
                                        AIC=c(mean(RndModelAIC))))
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                        AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.9012132	577
O	0.9004436	2029
P	0.8055556	36
S	0.9133858	254
V	0.9712759	1497

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0099       0.2569
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4101 Residual
## Null Deviance:       2157
## Residual Deviance: 2099  AIC: 2103
## log likelihood:  -1049.512
## Nagelkerke R2:  0.034356
```

```
## % pres/err predicted correctly:  -550.9402
## % of predictable range [ (model-null)/(1-null) ]:  0.01358708
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      1.9759       0.1557
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4101 Residual
## Null Deviance:       2157
## Residual Deviance: 2129  AIC: 2133
## log likelihood:  -1064.41
## Nagelkerke R2:  0.01678058
## % pres/err predicted correctly:  -554.7429
## % of predictable range [ (model-null)/(1-null) ]:  0.006791008
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    1.937976    -0.003306     0.182078
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4100 Residual
## Null Deviance:       2157
## Residual Deviance: 2129  AIC: 2135
## log likelihood:  -1064.381
## Nagelkerke R2:  0.01681527
## % pres/err predicted correctly:  -554.6881
## % of predictable range [ (model-null)/(1-null) ]:  0.006889006
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      2.6446      -0.4071
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4101 Residual
## Null Deviance:       2157
## Residual Deviance: 2140  AIC: 2144
## log likelihood:  -1070.084
## Nagelkerke R2:  0.01005394
## % pres/err predicted correctly:  -555.5421
## % of predictable range [ (model-null)/(1-null) ]:  0.005362711
## ***************************
```

```
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.533
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4102 Residual
## Null Deviance:       2157
## Residual Deviance: 2157  AIC: 2159
## log likelihood:  -1078.535
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -558.5428
## % of predictable range [ (model-null)/(1-null) ]:  0
## **************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    2.503490     0.003786
##
## Degrees of Freedom: 4102 Total (i.e. Null);  4101 Residual
## Null Deviance:       2157
## Residual Deviance: 2157  AIC: 2161
## log likelihood:  -1078.529
## Nagelkerke R2:  6.856254e-06
## % pres/err predicted correctly:  -558.5413
## % of predictable range [ (model-null)/(1-null) ]:  2.653663e-06
## **************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	2103.024	0.00000	1e+00	0.9999996	0.0343562	2.009886	0.2568571	NA	NA	NA	NA
preserved ~ pos	2132.821	29.79650	3e-07	0.0000003	0.0167806	6.975852	NA	NA	NA	0.1557195	NA
preserved ~ (I(pos^2) + pos)	2134.762	31.73789	1e-07	0.0000001	0.0168153	3.937976	NA	NA	-0.0033056	0.1820775	NA
preserved ~ CumErr	2144.168	41.14349	0e+00	0.0000000	0.0100539	2.644597	NA	-0.4070663	NA	NA	NA
preserved ~ 1	2159.069	56.04493	0e+00	0.0000000	0.0000000	2.532592	NA	NA	NA	NA	NA
preserved ~ stimlen	2161.058	58.03343	0e+00	0.0000000	0.0000069	2.503490	NA	NA	NA	NA	0.0037857

```r
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0226       0.3491
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1779
## Residual Deviance: 1716  AIC: 1720
## log likelihood:  -858.1795
## Nagelkerke R2:  0.04460052
## % pres/err predicted correctly:  -447.0249
## % of predictable range [ (model-null)/(1-null) ]:  0.01738777
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      1.9772       0.1768
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1779
## Residual Deviance: 1748  AIC: 1752
## log likelihood:  -874.1837
## Nagelkerke R2:  0.02199389
## % pres/err predicted correctly:  -450.9516
## % of predictable range [ (model-null)/(1-null) ]:  0.008775815
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     1.84497     -0.01231      0.27367
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3523 Residual
## Null Deviance:       1779
## Residual Deviance: 1748  AIC: 1754
## log likelihood:  -873.8392
## Nagelkerke R2:  0.02248267
## % pres/err predicted correctly:  -450.6888
## % of predictable range [ (model-null)/(1-null) ]:  0.009352058
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      2.6917      -0.4276
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1779
## Residual Deviance: 1767  AIC: 1771
## log likelihood:  -883.6102
## Nagelkerke R2:  0.00858224
## % pres/err predicted correctly:  -453.0083
## % of predictable range [ (model-null)/(1-null) ]:  0.004264976
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.595
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3525 Residual
## Null Deviance:       1779
## Residual Deviance: 1779  AIC: 1781
## log likelihood:  -889.6159
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -454.9529
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     2.70105     -0.01384
```

```
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1779
## Residual Deviance: 1779  AIC: 1783
## log likelihood:  -889.5529
## Nagelkerke R2:  9.014433e-05
## % pres/err predicted correctly:  -454.9372
## % of predictable range [ (model-null)/(1-null) ]:  3.437762e-05
## ***************************
```

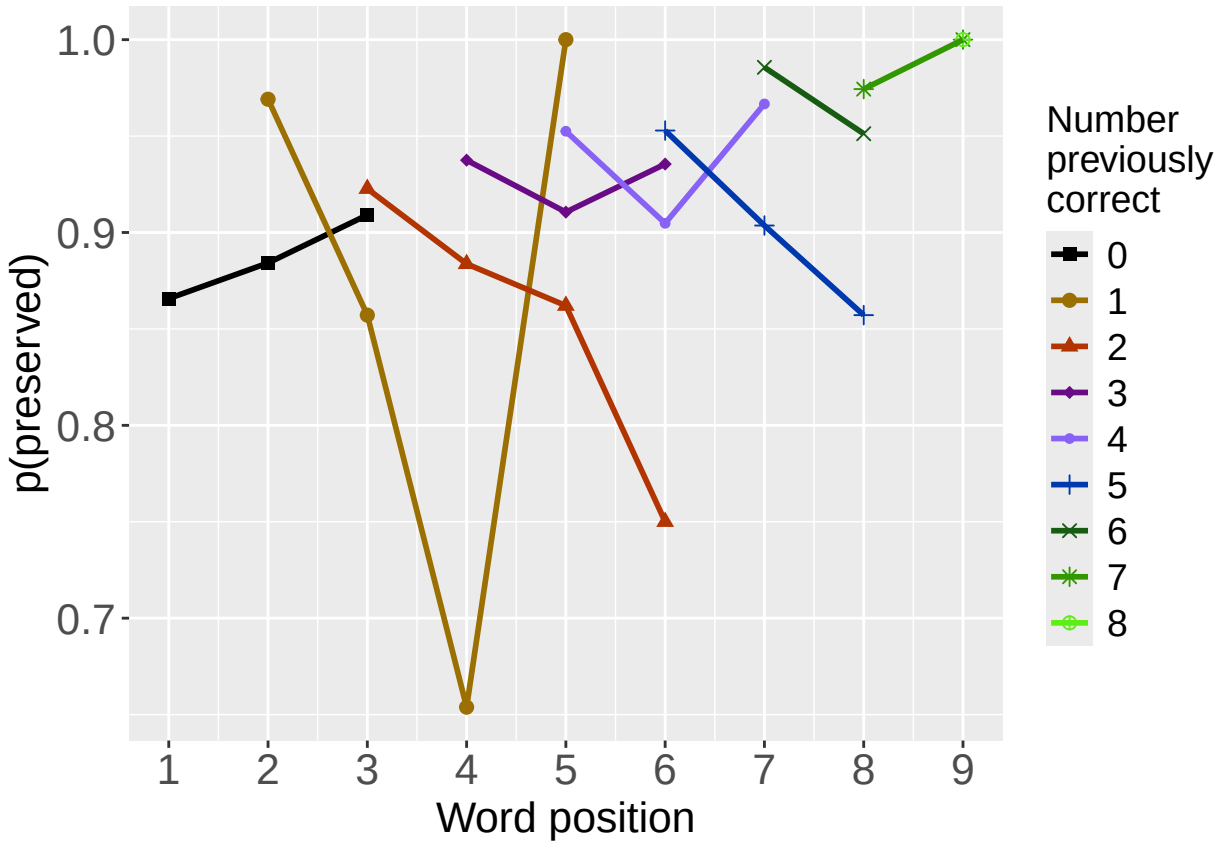
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(posˆ2)	pos	stimlen
preserved ~ CumPres	1720.359	0.00000	1e+00	0.9999998	0.0446002	3.022630	0.3491063	NA	NA	NA	NA
preserved ~ pos	1752.367	32.00842	1e-07	0.0000001	0.0219939	1.977163	NA	NA	NA	0.1768082	NA
preserved ~ (I(posˆ2) + pos)	1753.678	33.31943	1e-07	0.0000001	0.0224827	1.844970	NA	NA	-0.0123068	0.2736672	NA
preserved ~ CumErr	1771.226	50.86138	0e+00	0.0000000	0.0085822	2.691734	NA	-0.42761	NA	NA	NA
preserved ~ 1	1781.232	60.87288	0e+00	0.0000000	0.0000000	2.594645	NA	NA	NA	NA	NA
preserved ~ stimlen	1783.106	62.74693	0e+00	0.0000000	0.0000902	2.701048	NA	NA	NA	NA	-0.0138361

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

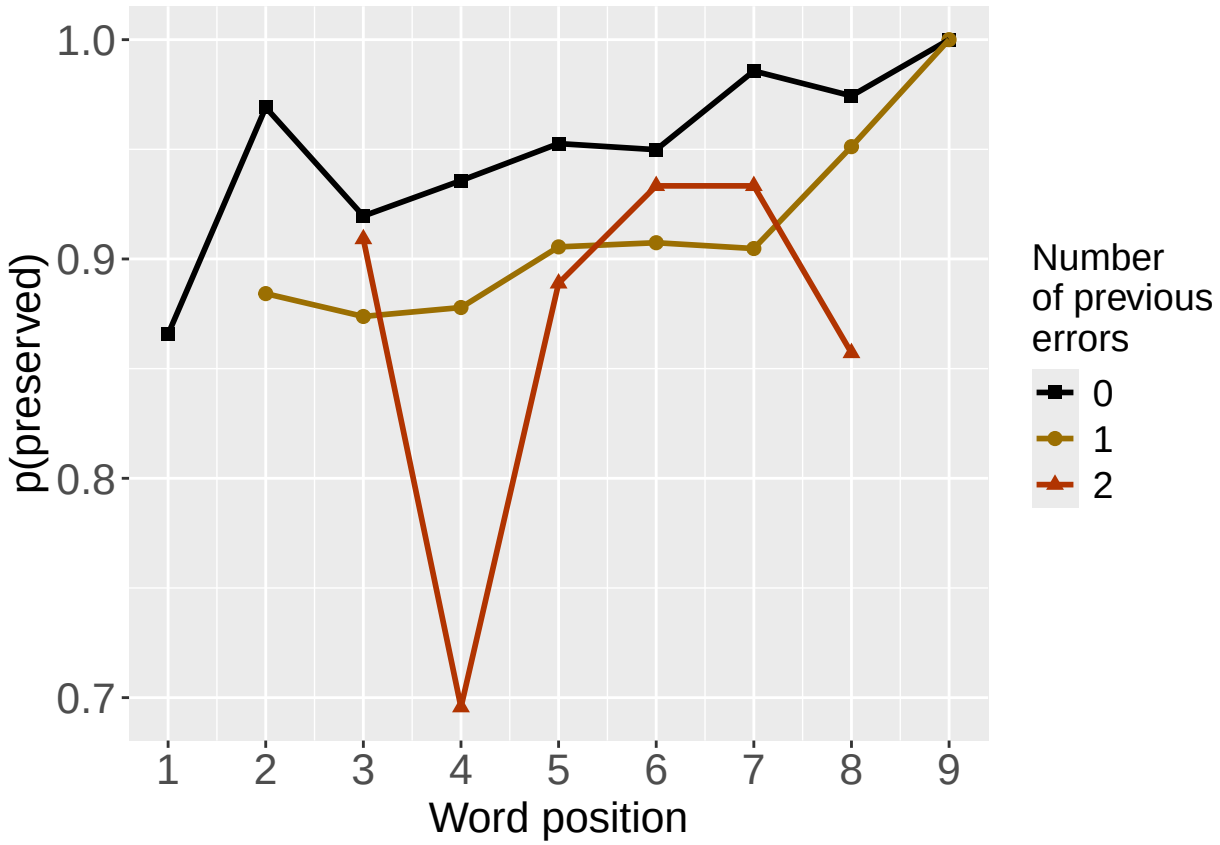
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

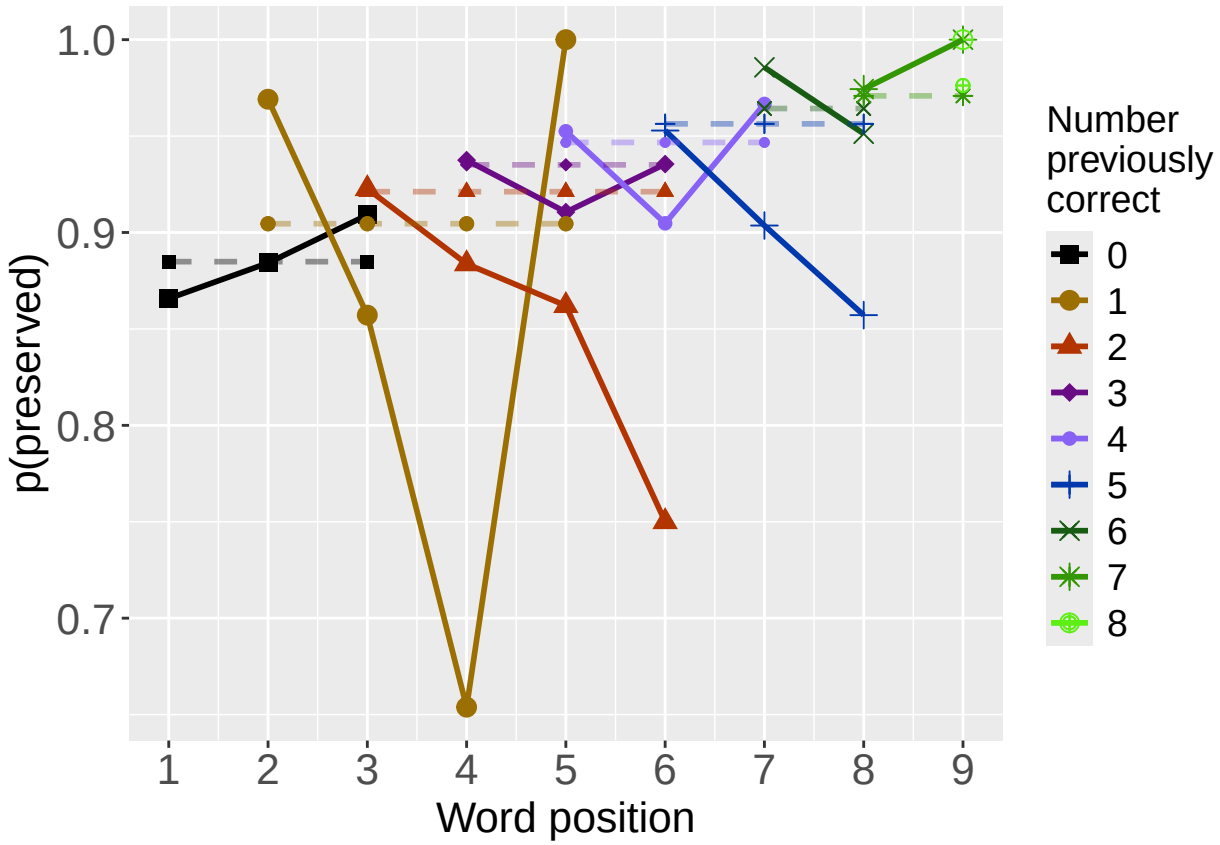
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

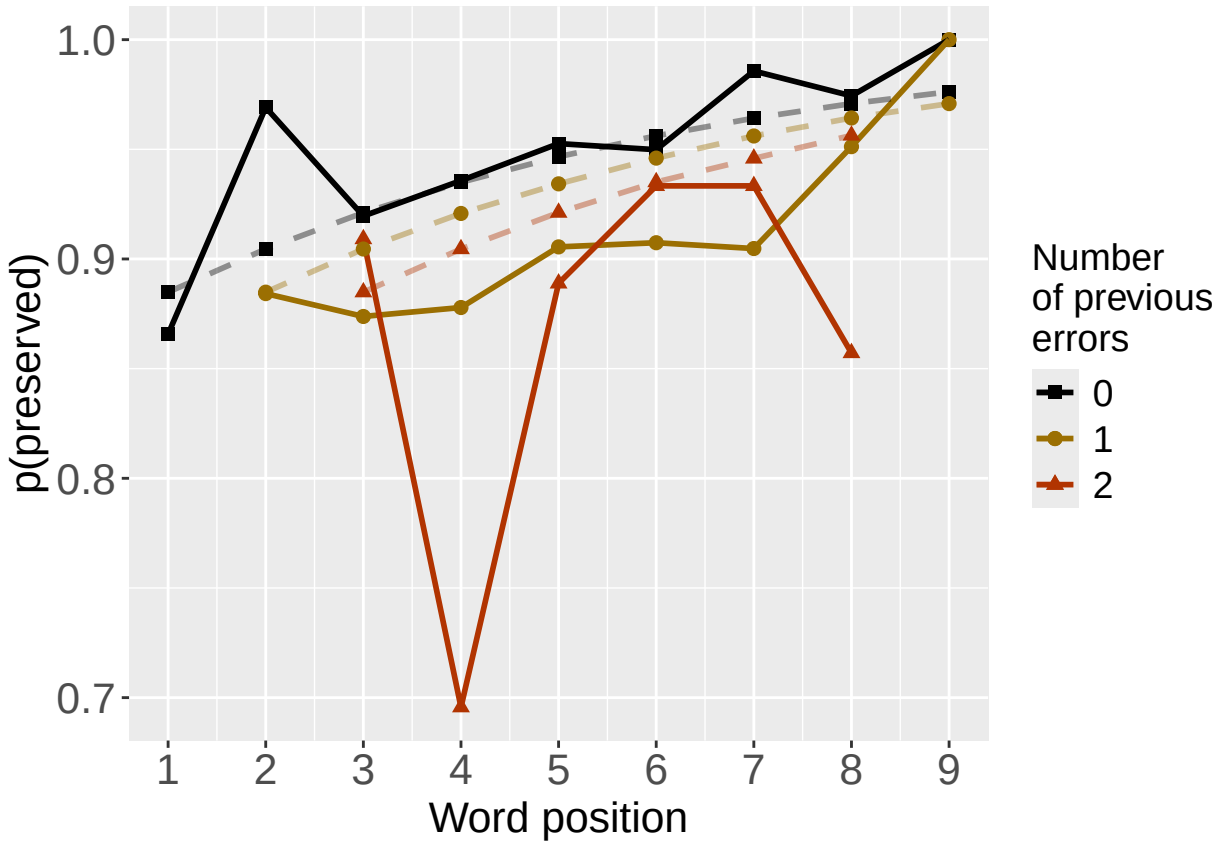
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
CumAICSummary <- NULL
```

```
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres     I(pos^2)          pos
##     2.55194      0.68120     -0.00571     -0.41188
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2270  AIC: 2278
## log likelihood:  -1134.869
## Nagelkerke R2:  0.04270474
## % pres/err predicted correctly:  -599.0639
## % of predictable range [ (model-null)/(1-null) ]:  0.01969604
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0394       0.2094
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2300  AIC: 2304
## log likelihood:  -1150.244
## Nagelkerke R2:  0.02603934
## % pres/err predicted correctly:  -604.7994
## % of predictable range [ (model-null)/(1-null) ]:  0.01032608
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    1.955624    -0.005146     0.178486
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2330
## log likelihood:  -1161.918
## Nagelkerke R2:  0.01330815
## % pres/err predicted correctly:  -607.7029
## % of predictable range [ (model-null)/(1-null) ]:  0.005582794
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	I(pos^2)	pos
preserved ~ CumPres + I(pos^2) + pos	2277.738	0.00000	1.0e+00	0.9999984	0.0427047	2.551945	0.6812033	-0.0057104	-0.4118758
preserved ~ CumPres	2304.489	26.75041	1.6e-06	0.0000016	0.0260393	2.039361	0.2094239	NA	NA
preserved ~ I(pos^2) + pos	2329.836	52.09737	0.0e+00	0.0000000	0.0133081	1.955624	NA	-0.0051456	0.1784864

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      stimlen
##     2.63649      0.23183     -0.08445
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2295  AIC: 2301
## log likelihood:  -1147.363
## Nagelkerke R2:  0.02917152
## % pres/err predicted correctly:  -604.1107
## % of predictable range [ (model-null)/(1-null) ]:  0.01145122
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0394       0.2094
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2300  AIC: 2304
## log likelihood:  -1150.244
## Nagelkerke R2:  0.02603934
## % pres/err predicted correctly:  -604.7994
## % of predictable range [ (model-null)/(1-null) ]:  0.01032608
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##    2.484930     0.002913
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2352
## log likelihood:  -1174.051
## Nagelkerke R2:  4.082105e-06
## % pres/err predicted correctly:  -611.1192
## % of predictable range [ (model-null)/(1-null) ]:  1.604814e-06
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	stimlen
preserved ~ CumPres + stimlen	2300.726	0.000000	1.0000000	0.8677832	0.0291715	2.636486	0.2318313	-0.0844504
preserved ~ CumPres	2304.489	3.762997	0.1523616	0.1322168	0.0260393	2.039361	0.2094239	NA
preserved ~ stimlen	2352.101	51.375630	0.0000000	0.0000000	0.0000041	2.484930	NA	0.0029129

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos
##      2.6189       0.6820      -0.4583
##
```

```
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2270  AIC: 2276
## log likelihood:  -1134.964
## Nagelkerke R2:  0.0426028
## % pres/err predicted correctly:  -599.2168
## % of predictable range [ (model-null)/(1-null) ]:  0.0194462
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0394       0.2094
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2300  AIC: 2304
## log likelihood:  -1150.244
## Nagelkerke R2:  0.02603934
## % pres/err predicted correctly:  -604.7994
## % of predictable range [ (model-null)/(1-null) ]:  0.01032608
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##       2.016        0.137
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2324  AIC: 2328
## log likelihood:  -1161.997
## Nagelkerke R2:  0.01322138
## % pres/err predicted correctly:  -607.796
## % of predictable range [ (model-null)/(1-null) ]:  0.005430539
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	pos
preserved ~ CumPres + pos	2275.927	0.00000	1e+00	0.9999994	0.0426028	2.618866	0.6819571	-0.4583000
preserved ~ CumPres	2304.489	28.56165	6e-07	0.0000006	0.0260393	2.039361	0.2094239	NA
preserved ~ pos	2327.994	52.06730	0e+00	0.0000000	0.0132214	2.016021	NA	0.1369989

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
```

```
                                "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	I(pos^2)	pos	stimlen
preserved ~ CumPres + pos	2275.927	0.000000	1.000000	0.999999	0.042602	8.618866	0.6819571	NA	-0.4583000	NA
preserved ~ CumPres + I(pos^2) + pos	2277.738	0.000000	1.000000	0.999998	0.042704	7.551945	0.6812033	-0.0057104	-0.4118758	NA
preserved ~ CumPres + stimlen	2300.726	0.000000	1.000000	0.867783	0.029171	3.636486	0.2318313	NA	NA	-0.0844504
preserved ~ CumPres	2304.489	26.750408	0.000001	0.000001	0.026039	3.039361	0.2094239	NA	NA	NA
preserved ~ CumPres	2304.489	3.762997	0.152361	0.132216	0.026039	3.039361	0.2094239	NA	NA	NA
preserved ~ CumPres	2304.489	28.561652	0.000000	0.000000	0.026039	3.039361	0.2094239	NA	NA	NA
preserved ~ pos	2327.994	52.067300	0.000000	0.000000	0.013221	4.016021	NA	NA	0.1369989	NA
preserved ~ I(pos^2) + pos	2329.836	52.097370	0.000000	0.000000	0.013308	1.955624	NA	-0.0051456	0.1784864	NA
preserved ~ stimlen	2352.101	51.375630	0.000000	0.000000	0.000004	2.484930	NA	NA	NA	0.0029129

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos     log_freq
```

```
##      2.5652       0.6413      -0.4103       0.1292
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2252  AIC: 2260
## log likelihood:  -1125.808
## Nagelkerke R2:  0.05247175
## % pres/err predicted correctly:  -596.3283
## % of predictable range [ (model-null)/(1-null) ]:  0.02416494
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos      stimlen     log_freq
##     2.78128      0.64558     -0.40661     -0.03132      0.12202
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4388 Residual
## Null Deviance:       2348
## Residual Deviance: 2251  AIC: 2261
## log likelihood:  -1125.461
## Nagelkerke R2:  0.05284458
## % pres/err predicted correctly:  -596.2657
## % of predictable range [ (model-null)/(1-null) ]:  0.02426733
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos      stimlen
##      3.0986       0.6862      -0.4433      -0.0708
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2266  AIC: 2274
## log likelihood:  -1133.02
## Nagelkerke R2:  0.04470161
## % pres/err predicted correctly:  -598.6436
## % of predictable range [ (model-null)/(1-null) ]:  0.0203825
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos
##      2.6189       0.6820      -0.4583
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
```

```
## Null Deviance:       2348
## Residual Deviance: 2270  AIC: 2276
## log likelihood:  -1134.964
## Nagelkerke R2:  0.0426028
## % pres/err predicted correctly:  -599.2168
## % of predictable range [ (model-null)/(1-null) ]:  0.0194462
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.507
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4392 Residual
## Null Deviance:       2348
## Residual Deviance: 2348  AIC: 2350
## log likelihood:  -1174.054
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -611.1202
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
                       row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	pos	log_freq	stimlen
preserved ~ CumPres + pos + log_freq	2259.616	0.000000	1.000000	0.6573283	0.0524712	2.565157	0.6413383	-0.4102580	0.1291964	NA
preserved ~ CumPres + pos + stimlen + log_freq	2260.923	1.306758	0.5202848	0.3419979	0.0528446	2.781283	0.6455789	-0.4066132	0.1220218	-0.0313203
preserved ~ CumPres + pos + stimlen	2274.039	14.423259	0.0007380	0.0004851	0.0447016	6.098627	0.6862114	-0.4432831	NA	-0.0708043

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	pos	log_freq	stimlen
preserved ~ CumPres + pos	2275.927	16.311057	0.000287	1.000188	7.042602	2.618866	0.681957	-0.458300	NA	NA
preserved ~ 1	2350.109	90.492767	0.000000	0.000000	0.000000	2.507312	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

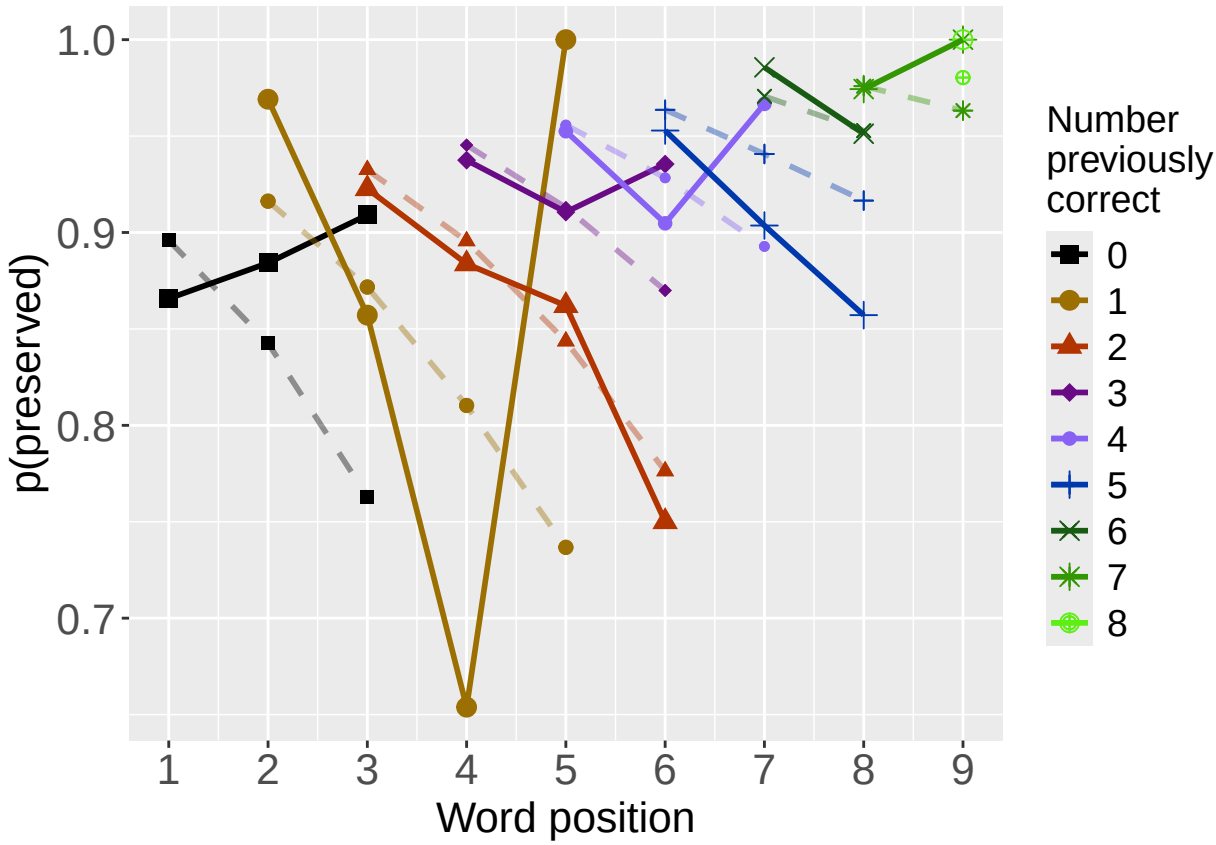
```
## Single term deletions
##
## Model:
## preserved ~ CumPres + pos + log_freq
##          Df Deviance    AIC
## CumPres   1   2298.4 2304.4
## pos       1   2275.5 2281.5
## log_freq  1   2269.9 2275.9
## <none>        2251.6 2259.6
```

```
################################
# Single deletions from best model
################################

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

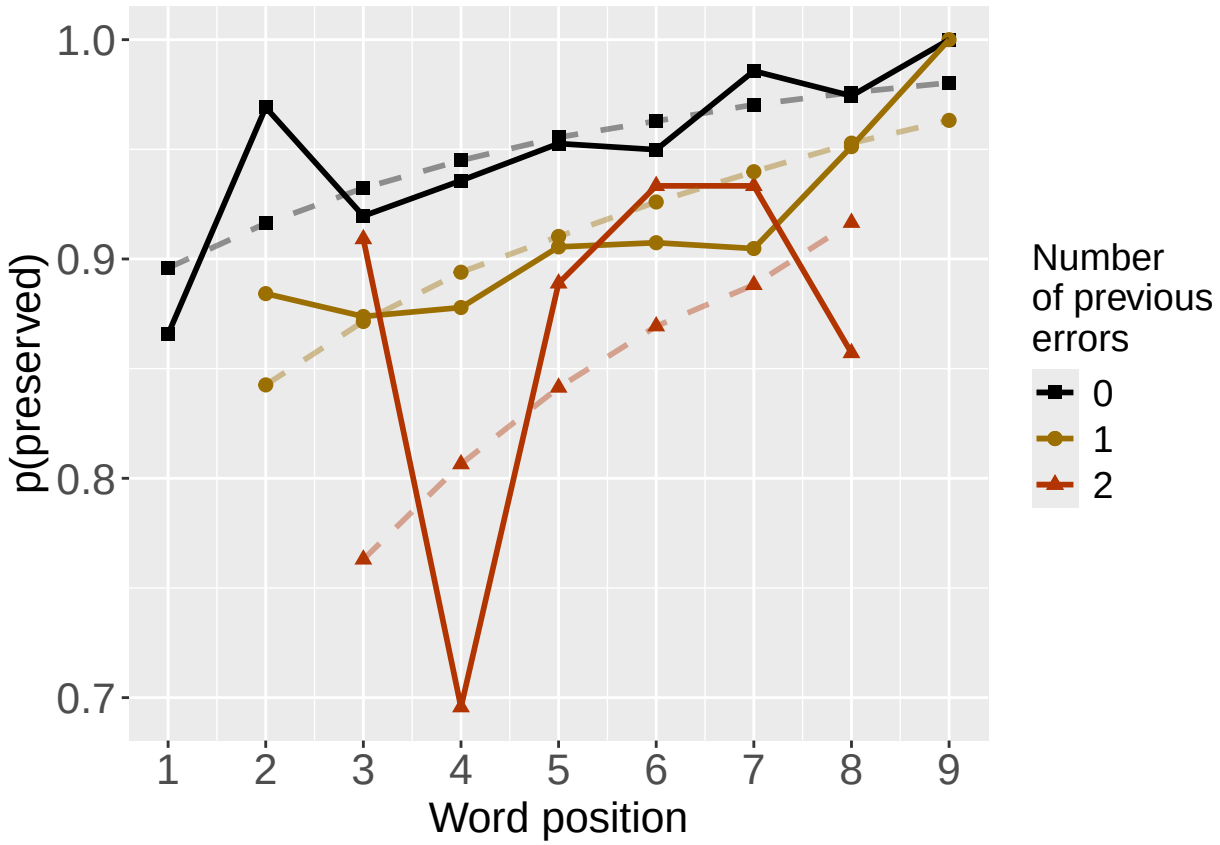
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

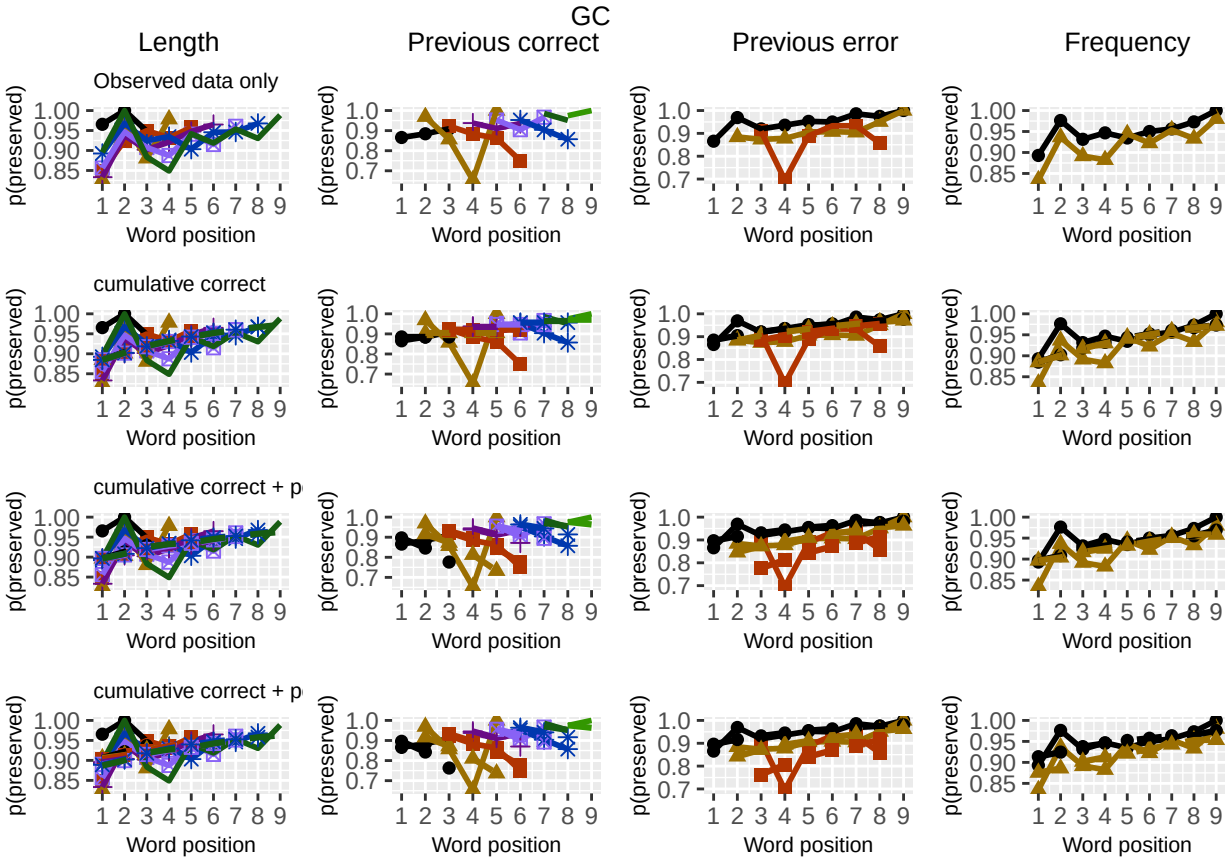
```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0394       0.2094
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4391 Residual
## Null Deviance:       2348
## Residual Deviance: 2300  AIC: 2304
## log likelihood:  -1150.244
## Nagelkerke R2:  0.02603934
## % pres/err predicted correctly:  -604.7994
## % of predictable range [ (model-null)/(1-null) ]:  0.01032608
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos
##      2.6189       0.6820      -0.4583
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4390 Residual
## Null Deviance:       2348
## Residual Deviance: 2270  AIC: 2276
## log likelihood:  -1134.964
## Nagelkerke R2:  0.0426028
## % pres/err predicted correctly:  -599.2168
## % of predictable range [ (model-null)/(1-null) ]:  0.0194462
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos     log_freq
##      2.5652       0.6413      -0.4103       0.1292
##
## Degrees of Freedom: 4392 Total (i.e. Null);  4389 Residual
## Null Deviance:       2348
## Residual Deviance: 2252  AIC: 2260
## log likelihood:  -1125.808
## Nagelkerke R2:  0.05247175
## % pres/err predicted correctly:  -596.3283
## % of predictable range [ (model-null)/(1-null) ]:  0.02416494
## ***************************

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
```

```
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
```

```r
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumPres	pos	log_freq
McFadden	0.0209521	0.0110696	0.0090721
SquaredCorrelation	0.0110765	0.0058519	0.0047973
Nagelkerke	0.0110765	0.0058519	0.0047973
Estrella	0.0113088	0.0059750	0.0048957

	deviance	deviance_explained
CumPres + pos + log_freq	2251.616	96.49277
CumPres + pos	2269.927	78.18171
CumPres	2300.489	47.62006
null	2348.109	0.00000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumPres + pos + log_freq	4.109382	100.00000	18.97661
CumPres + pos	3.329561	81.02339	31.67248
CumPres	2.028017	49.35091	49.35091
null	0.000000	NA	0.00000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                          model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumPres + pos + log_freq CumPres + pos + log_freq 2251.616          96.49277          4.109382                     100.00000
## CumPres + pos                       CumPres + pos 2269.927          78.18171          3.329561                      81.02339
## CumPres                                   CumPres 2300.489          47.62006          2.028017                      49.35091
## null                                         null 2348.109           0.00000          0.000000                            NA
##                            increment_in_explained
## CumPres + pos + log_freq                 18.97661
## CumPres + pos                            31.67248
## CumPres                                  49.35091
## null                                      0.00000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##          Nagelkerke
## CumPres   0.5098336
## pos       0.2693543
## log_freq  0.2208121
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

model	p_accounted_for	model_deviance
preserved ~ CumPres+pos+log_freq	0.6137370	2251.616
preserved ~ CumPres+pos	0.6276731	2269.927
preserved ~ CumPres	0.6340626	2300.489

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                             model p_accounted_for model_deviance diff_CumPres+pos+log_freq diff_CumPres+pos diff_CumPres
## 1 preserved ~ CumPres+pos+log_freq       0.6137370       2251.616                0.00000000     -0.013936178 -0.020325669
## 2          preserved ~ CumPres+pos       0.6276731       2269.927                0.01393618      0.000000000 -0.006389491
## 3              preserved ~ CumPres       0.6340626       2300.489                0.02032567      0.006389491  0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumPres+pos+log_freq	diff_CumPres+pos	diff_CumPres
preserved ~ CumPres+pos+log_freq	0.0000000	-0.0139362	-0.0203257
preserved ~ CumPres+pos	0.0139362	0.0000000	-0.0063895
preserved ~ CumPres	0.0203257	0.0063895	0.0000000